



Übungsblatt: Privacy Preserving Computation

Ansatz: Differential Privacy

Ziel der Übung

Ziel dieser Übung ist es, praxisnahe IoT-Anwendungsfälle systematisch zu analysieren und Beispiele für Privacy-Preserving Computation durchzuspielen.

Use-Case: Der neugierige Smart-Meter-Server

Stellen Sie sich ein Smart Grid vor: Ihre IoT-Stromzähler senden stündlich Verbrauchsdaten an einen zentralen Server. Der **neugierige Server** (z.B. Energieversorger oder Hacker) erhält diese Rohdaten und kann daraus ableiten:

- Wann Sie zu Hause sind (Verbrauchsspitzen)
- Welche Geräte Sie nutzen (typische Lastkurven)
- Ihren Lebensstil (Urlaubszeiten)

Differential Privacy löst das Problem: Jeder Zähler fügt **kontrolliertes Rauschen** hinzu. Der Server erhält nutzbare Statistiken (z.B. Stadtweiter Durchschnittsverbrauch), kann aber keine Rückschlüsse auf **Einzelhaushalte** ziehen.

Der Grundgedanke ist, dass Sie jeden numerischen Datensatz mit einem Rauschen belegen, bevor sie ihn „hochladen“. D.h. statt des Wertes 15, übermitteln Sie den Wert $15 + \text{ZUFALL}(-20 \text{ bis } 20)$

Ihre Aufgabe: Simulieren Sie in Excel, wie sich die Genauigkeit der Durchschnittsstatistik mit steigendem Noise Level verhält.

Aufgabe: Noise-Injection-Simulation

Erstellen Sie in Excel/LibreOffice eine Tabelle mit **100 Zufallswerten** (Zeilen 5–104). Lassen Sie die oberen Zeilen frei, für Header (sinnvolle Überschrift), Parameter und Durchschnittswerte

Spalte A: Aufsteigende Nummer 1,2,3,4 ...

Spalte B: Zufallswert 0 bis 1

Spalte C: Zufallszahlen von 0 bis 99

→ Header: Durchschnittswert (der Spalte)

Spalte D: Grundete Zahlen aus B

→ Header: Durchschnittswert (der Spalte)

Noise Block $r=20$

Spalte F: positive oder negative Zufallszahl von $-r$ bis r → Parameter $r=20$, Durchschnittswert

Spalte G: Spalte C + Spalte F → Durchschnittswert

Spalte H: gerundete Spalte G → Durchschnittswert

Weitere solche Noise Blöcke für $r = 5$ und $r = 100$

Zusatz: $N=1000$ (neue Tabelle), Noise=100 – beobachten Sie die Konvergenz.

Analyse & Diskussion

Noise-Effekt: Bei welchem Noise Level (5/20/100) bleibt der Durchschnitt stabil, aber die Einzelwerte unkenntlich?

Beobachtung: Wie nähert sich der verrauschte Durchschnitt dem Original bei steigendem N (100 vs. 1000)? Warum?

Trade-off: Noise schützt Privatsphäre, erhöht aber die Varianz. Diskutieren Sie den **Epsilon-Parameter** (ϵ): Kleines ϵ = viel Noise = hoher Schutz, aber ungenaue Statistiken.

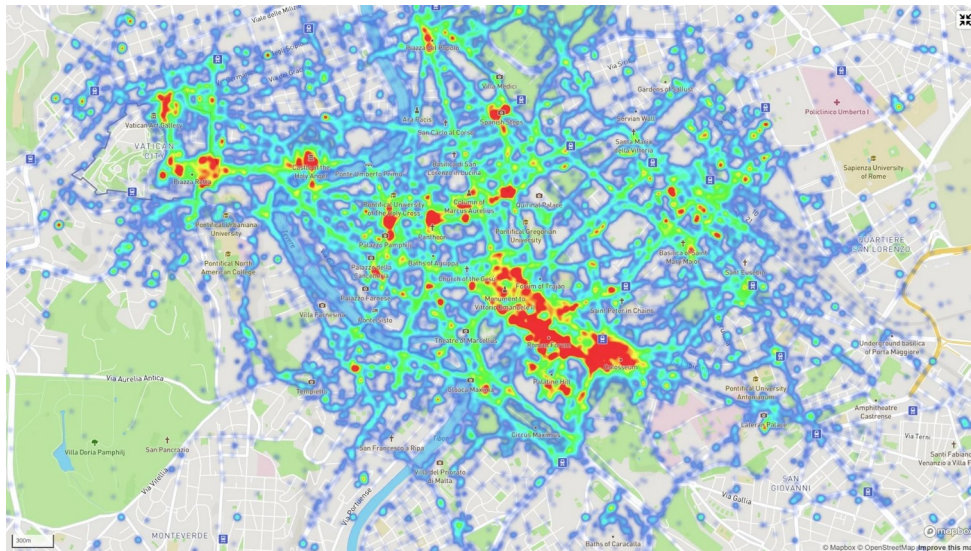
Anwendungsfälle: Für welchen Use-Case aus dem IoT-Kontext empfehlen Sie welches Noise Level?

- **Smart Meter:** Stadtweiter Verbrauch ohne Haushaltsprofile.
- **Mobilität (Uber):** Durchschnittliche Fahrtlängen ohne Einzelstrecken.
- **Biomedizin:** Aggregierte Gesundheitsdaten ohne Patientenidentifikation.
- **Social Media:** Trendanalysen ohne Profilzuordnung.

Diskussion:

Sehen Sie eine Möglichkeit um dieses System zu nutzen um eine Heatmap für die Position von Autos in einer Stadt zu erstellen, ohne zu offenbaren wo einzelne Autos sind?

Beispiel:



Ansatz: Secure Multiparty Computation

Ziel der Übung:

Sie lernen, wie mehrere Parteien eine gemeinsame Statistik berechnen können, ohne dass jemand die individuellen Werte erfährt.

Use-Case:

Ein Informatikprofessor möchte zum Alter seiner Studenten den Durchschnitt und die Summe berechnen. Niemand soll jedoch die Altersangaben sehen.

Schritt 1: Shamir's Secret Sharing

Ihre Rollen:

- Server 1, 2, 3: Sie sind die „vertrauenswürdigen Rechenzentren“.
- Dateneigentümer 4–10: Jeder von Ihnen wählt ein Geheimnis (z. B. Ihr Alter).

Aufgabe für Dateneigentümer:

1. Nehmen Sie 3 Zettel und teilen Sie Ihr Geheimnis in 3 Summanden auf, sodass sie sich wiederum ergeben.
Beispiel: Alter = 25 → Zettel: -17, +30, +12 (da $-17 + 30 + 12 = 25$).
2. Geben Sie einen Zettel pro Server (anonym, ohne Kommentar).

Aufgabe für Server:

Addieren Sie alle erhaltenen Zettel auf Ihrem Block. Notieren Sie die Zwischensumme.

Diskussionsfrage:

Was wissen die Server jetzt über die individuellen Alter?

Schritt 2: Ring-Addition (Multi-Party-Computation-Protokoll)

Aufgabe für Server:

Server 1 addiert zu seiner Zwischensumme eine große Zufallszahl X (z. B. 1.000.000).

Server 1 gibt die neue Summe ($X + \text{Summe}_1$) an Server 2 weiter.

Server 2 addiert seine Zwischensumme und gibt die Gesamtsumme an Server 3.

Server 3 addiert seine Zwischensumme und gibt die Gesamtsumme zurück an Server 1.

Server 1 zieht X ab und teilt das Ergebnis durch die Anzahl der Dateneigentümer.

Was ist das Ergebnis?

Diskussionsfragen:

- Warum können die Server nichts über individuelle Alter lernen?
- Was passiert, wenn einer oder zwei Server böswillig handelt?
Welche Schutzziele werden verletzt?
- Vergleich: Wie unterscheidet sich dieses Protokoll von einem zentralen Server?

Ansatz: Homomorphic Encryption

Ziel der Übung:

Sie berechnen mit einfacher exponentieller Verschlüsselung, wie die Bank den Kontostand ermittelt, ohne Alice' Transaktionen zu kennen.

Use-Case:

Alice nutzt eine Taschengeld-App um ihre Taschengeldeingänge und ihre Galdabgänge (am Schulkiosk) zu tracken. Dazu nutzt sie verschiedene Geräte, Handy, Tablet, PC. Die Daten sollen aber **verschlüsselt (!)**, **zentral** von einem neugierigen Datenhost synchron gespeichert werden.

Alice möchte aber nicht, dass der Datenhost sieht, welche Beträge bei ihr ein und ausgehen.

Alice nutzt Homorphic Encryption, d.h. sie verschlüsselt alle Ein- und Abgangswerte, die sie der Bank schickt. Die Bank soll jedoch dennoch den richtigen Wert stets aktuell halten, ohne ihn im Klartext zu sehen.

Alice' Vorgehen: Die Verschlüsselung der Werte vor dem Versenden

Alice' Regel: Ein Wert m wird als $\text{Enc}(m) = 2^m$ verschlüsselt.

Beispiele:

$$m = 3 \rightarrow \text{Enc}(3) = 2^3 = 8$$

$$m = 2 \rightarrow \text{Enc}(2) = 2^2 = 4$$

$$m = -4 \rightarrow \text{Enc}(-4) = 2^{-4} = 0,25$$

Eigenschaft: $\text{Enc}(a) \times \text{Enc}(b) = 2^a \times 2^b = 2^{(a+b)} = \text{Enc}(a+b)$

Eine Addition in der Alice - Welt entspricht einer Multiplikation in der Datenhost - Welt.

Vorgehen der Bank:

Annahme: Datenhost kennt den Schlüssel ($k=2$) nicht und Alice hat den Wert 2^0 initial übertragen

Sei T der aktuelle Taschengeldstand der im Datenhost gespeichert ist.

Jeder eingehende Wert X wird zu T multipliziert. $T := T * X$

Alice' Taschengeld-Transaktionen - Ihre Aufgabe: Berechnen Sie Schritt für Schritt!

Alice hat nun folgende Transaktionen:

+ 1 €, + 3 €, + 2 €, - 4 €, + 10 €, 0 €, - 1 €

- Welche Werte übermittelt Sie an den Datenhost ?
- Was ist der Wert beim Datenhost ?
- Wenn Sie den aktuellen "Kontostand" vom Datenhost abrufen, wie kann sie den Wert in Klartext sehen? Welche Operation nutzt Sie zum Entschlüsseln?
- Welche Information hat der Datenhost aus den Transaktionen schlussfolgern?
- Welche Funktion bietet der Ansatz? Welche Schwächen hat er?